

Reviewer Pack — Minimal Rank of Noncrossing Partitions vs Tree-width of Reso...

Entry ce06bfd85c84 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 19:39:51 UTC

Minimal Rank of Noncrossing Partitions vs Tree-width of Resolution Proofs

Entry ID: ce06bfd85c84 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 19:39:51 UTC

1. Conjecture

Field A (mathematical branch): Enumerative Combinatorics (Noncrossing Partitions) **Field B** (complexity object): Complexity Theory: Resolution Proof Tree-width

Statement:

[‘For all k -CNF formulas, the minimal rank of a noncrossing partition of their incidence poset is linearly related to the tree-width of their resolution proof.’, ‘More precisely, for every k -CNF formula F with n variables and m clauses, there exist constants $c_1, c_2 > 0$ such that $c_1 * \min_rank(\text{noncrossing_partition}(F)) \leq \text{tree_width}(\text{resolution_proof}(F)) \leq c_2 * \min_rank(\text{noncrossing_partition}(F))$.’]

Rationale (proposer’s reasoning):

[‘Noncrossing partitions provide a combinatorial structure for counting the number of paths in a graph, which can be used to bound the size of the resolution proof. The connection suggests that the complexity of proving a CNF formula may be related to its combinatorial properties.’, ‘The minimal rank of a noncrossing partition represents a certain level of simplicity or complexity in the arrangement of clauses and variables in the incidence poset, which could reflect the inherent difficulty of resolving the formula.’]

Taxonomy category: Enumerative Combinatorics (Noncrossing Partitions) × Complexity Theory: Resolution Proof Tree-width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c27f2743ff328032

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The relationship between the minimal rank of a noncrossing partition and the tree-width of a resolution proof for k -CNF formulas is supported if at least 90% of the computed pairs $(\min_rank, \text{tree_width})$ fall within the bounds $c_1 * \min_rank \leq \text{tree_width} \leq c_2 * \min_rank$ with $c_1, c_2 > 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, m):
        variables = set(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, random.randint(1, n))
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return variables, clauses

    def noncrossing_partition_rank(variables, clauses):
        # Simplified rank calculation (example implementation)
        return len(variables) + len(clauses)

    def resolution_proof_tree_width(variables, clauses):
        # Simplified tree-width calculation (example implementation)
        return max(len(c) for c in clauses)

    n = random.randint(5, 40)
    m = random.randint(n, 2 * n)
    variables, clauses = generate_k_cnf(n, m)

    rank = noncrossing_partition_rank(variables, clauses)
    tree_width = resolution_proof_tree_width(variables, clauses)

    if tree_width == 0:
```

```

    return {
        "metric_name": "Rank vs Width",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Tree-width is zero"
    }

c1 = Fraction(1, 2)
c2 = Fraction(3, 2)

holds = c1 * rank <= tree_width <= c2 * rank

    return {
        "metric_name": "Rank vs Width",
        "metric_value": tree_width,
        "instances_tested": 1,
        "conjecture_holds": holds,
        "counterexample": "" if holds else f"Counterexample: rank={rank}, tree-width={tree_width}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(v is not None for v in metric_values):
        mean = sum(metric_values) / len(metric_values)
        std_dev = math.sqrt(sum((v - mean) ** 2 for v in metric_values) / len(metric_values))
        if support_fraction >= 0.9:
            print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
        else:
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed=None")
    else:
        print("RESULT: INCONCLUSIVE reason=metric_saturation_or_division_by_zero")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_42b0aa27.py", line 73, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_42b0aa27.py", line 41, in run_trial

```

```

    variables, clauses = generate_k_cnf(n, m)
                             ~~~~~^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_42b0aa27.py", line 25, in generate_k_cnf
    clause = random.sample(variables, random.randint(1, n))
               ~~~~~^
                             ~~~~~^
File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
TypeError: Population must be a sequence.  For dicts or sets, use sorted(d).

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_42b0aa27.py", line 25, in generate_k_cn

File "/usr/lib/python3.12/random.py", line 413, in sample

```
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support condition. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12679
2	propose	ollama_remote	glm4:latest	0	0	11504
3	propose	ollama_remote	glm4:latest	0	0	10025
4	preregistration	ollama_remote	glm4:latest	0	0	5867
5	novelty	ollama_remote	glm4:latest	0	0	4861
6	novelty	ollama_remote	glm4:latest	0	0	5076
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11812
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8312
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7919
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8341
11	judge	ollama_remote	glm4:latest	0	0	11351

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97750 ms total latency. Provider mix: {'ollama remote': 11}

(full prompt+response transcripts available in `research/audit/ce06bfd85c84.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ce06bfd85c84.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ce06bfd85c84.tar.gz` (if generated)